WORKSHOP #2 - DESIGN ARTIFACTS AND SYSTEM MODELING

Alejandro Ramirez Nuñez
Juan Felipe Quiroga Medina
Juan Camilo Lopez Leon
Juan David Yopasa Maldonado

2026-2S

Ingeniería de Software II
Liliana Marcela Olarte

Universidad Nacional de Colombia
Facultad de Ingeniería
Sede Bogotá

1. System Domain

1.1 System Context

In Bogotá, citizens commonly perceive that public construction works miss their deadlines, and that information about their progress and effects on mobility is scattered or hard to find. This lack of accessible information reduces public trust in the district administration and limits citizens' ability to exercise oversight.

The Construction Works Oversight Website is a web platform that centralizes information about active public construction sites in Bogotá during 2026, with an initial scope of 200 construction sites. It presents each construction site on an interactive map together with its physical progress, performance indicators, budget and contractor and gives citizens tools to participate: rating and commenting on construction sites, comparing them, subscribing to updates, and preparing a right of petition.

System boundaries. The system *does*: display and consult construction site information; calculate performance indicators; manage user accounts and citizen participation; let the administrator manage construction site data and moderate content. The system *does not*: file rights of petition before public entities (it only provides a guide and a downloadable template); synchronize in real time with district information systems (data is loaded by the administrator from open data sources); or verify the legal identity of citizens.

External system	Interaction with the system
Open data sources (IDU, Secretaría de Movilidad de Bogota, SECOP II)	Provide the public information about construction sites (contract, budget, dates, progress, road closures), which the administrator loads into the system with its source and update date (NFR_14).

Email service	Sends password recovery messages (FR_18) and periodic updates to users subscribed to a construction site (FR_06).
Map provider	Supplies the base map of Bogotá on which construction sites and road closures are drawn (FR_01, FR_29).

1.2 Actor Characterization

Attribute	Visitor	Registered citizen	Administrator
Description	Any citizen who accesses the website without an account or without logging in.	Citizen who created an account with an email address and is logged in.	Member of the platform team responsible for the data and the content published on the site.
Goals	Find out which construction sites are active near them, their progress and road closures.	Participate in the oversight of construction sites: express opinions, follow specific sites and take formal action.	Keep construction site information accurate and up to date, and keep the discussion respectful and reliable.
Permissions	Read-only: map, detail sheet, search, filters, sorting, evolution graph, milestone timeline and road closures.	Everything a visitor can do, plus: rate, comment, delete own comments, report comments, compare, export to PDF, subscribe, download the	Add, edit and delete construction sites; delete any comment; access the administrator dashboard. Account is assigned by the system, not created

		petition template, and manage their profile.	through public registration (FR_30).
Expected frequency of use	Occasional, usually when a construction site affects their area or route.	Recurring, especially for subscribed or nearby construction sites.	Frequent: periodic data updates and daily review of reported comments.
Technical profile	General public with varied digital skills; accesses from mobile or desktop.	Same as the visitor.	Trained in the use of the administration functions and in reading the open data sources.
Related user stories	US-01, US-02, US-05, US-14, US-18, US-19, US-23, US-24	US-03, US-04, US-06, US-07, US-08, US-09, US-10, US-11, US-15, US-16, US-20, US-21, US-22	US-12, US-13, US-17, US-25

1.3 Business Rules

ID	Business rule	Related requirements
BR-01	Only registered and logged-in users can rate, comment, report comments, compare construction sites, export data, subscribe to updates and download the petition template.	FR_05, FR_06, FR_08, FR_09, FR_11, FR_14, FR_22
BR-02	A user must accept the personal data processing policy before their registration is completed; the acceptance and its date are recorded.	FR_17, NFR_13

BR-03	Each email address can be associated with only one user account.	FR_17
BR-04	Administrator accounts are created only by the system; public registration always creates a registered citizen account.	FR_30, NFR_15
BR-05	A registered user can give only one rating per construction site, with a value from 0 to 5 stars, and can update it. The construction site's satisfaction score is the average of its ratings.	FR_08
BR-06	A registered user can publish at most one comment per minute.	FR_09, NFR_04
BR-07	A user can delete only their own comments; the administrator can delete any comment.	FR_10, FR_12
BR-08	A user cannot report their own comments and can report the same comment only once.	FR_11
BR-09	A construction site is considered **active** when it has a start date, has not been marked as finished, and its physical progress is below 100%. Only active construction sites are shown on the map.	FR_01, FR_25
BR-10	The **performance index** of a construction site is the ratio between its actual physical progress and the progress planned for the current date. A value of 1 or more means the construction site is on schedule.	FR_04, FR_22, FR_23, FR_24
BR-11	Every construction site must record the source of its data and the date of its last update.	FR_26, NFR_14
BR-12	Each update of a construction site's physical progress is stored as a historical record; previous records are never overwritten, so that the evolution graph can be built.	FR_26, FR_27

BR-13	The system does not file rights of petition before public entities; it only provides a guide and a downloadable template that the user must complete and file directly with the competent entity.	FR_14, FR_15
BR-14	Subscribed users receive an email when the physical progress or the estimated end date of the construction site changes.	FR_06

2. CRC Cards

Identity service:

User *(abstract; specialized by Citizen and Administrator)*

Responsibilities	**Collaborators**
Knows its unique email (BR-03), hashed password (NFR_09), name, profile photo and description	ParticipationClient
Authenticates with its credentials and receives an access token (FR_16, FR_30)	
Updates its profile data and password (FR_20)	
Requests password recovery; the Identity service sends the recovery email (FR_18)	
Deletes its account and announces the deletion to the Participation service	

Citizen *(extends User)*

Responsibilities	Collaborators
Knows whether it accepted the personal data policy, and when (BR-02, NFR_13)	
Is referenced by its id in the ratings, comments, reports and subscriptions of the Participation service	

Administrator *(extends User; created only by the system (BR-04))*

Responsibilities	Collaborators
Has the administrator role that authorizes administration requests in every service (NFR_15)	
Is referenced by its id in progress records and resolved comment reports	

ParticipationClient *(non-persistent; one in the Identity service and one in the Construction Sites service)*

Responsibilities	Collaborators
Announces to the Participation service that a user account was deleted (Identity service)	ParticipationEventHandler (via Participation API)
Announces to the Participation service that the progress of a construction site changed, or that a construction site was deleted (Construction Sites service)	

Construction Sites service:

ConstructionSite

Responsibilities	**Collaborators**
Knows its name, description, type, geometry (point or line), budget, start date, contractual end date, estimated end date, current physical progress and status (FR_01, FR_04)	Locality
Knows its data source and last update date (BR-11)	Contractor
Knows its locality, contractor and responsible entity	PublicEntity
Determines whether it is active (BR-09)	ProgressRecord
Calculates its performance index (BR-10)	Milestone
Registers a new progress record without overwriting previous ones (BR-12)	RoadClosure
Provides its progress history for the evolution graph (FR_27)	SiteImage
Announces changes of its progress or estimated end date, and its deletion, to the Participation service (BR-14)	ParticipationClient

ProgressRecord

Responsibilities	**Collaborators**
Knows its date, physical progress percentage and estimated end date at that moment	ConstructionSite
Knows the id of the administrator who registered it	
Validates that the progress is between 0 and 100	

Provides a data point for the evolution graph (FR_27)	

PublicEntity

Responsibilities	**Collaborators**
Knows its name, acronym, address and petition channel	ConstructionSite
Knows the construction sites it is responsible for	PetitionTemplate (via Construction Sites API)
Provides its recipient data for the petition template (FR_15)	

Participation service:

Rating

Responsibilities	**Collaborators**
Knows its score, dates, citizen id and construction site id	SitesClient
Validates that the score is between 0 and 5 (FR_08)	
Ensures one rating per citizen per construction site, and allows updating it (BR-05)	
Calculates the average rating of a construction site (BR-05)	
Verifies that the rated construction site exists	

Comment

Responsibilities	Collaborators
Knows its text, publication date, author id and construction site id (FR_09)	CommentReport
Verifies that its author has not published another comment in the last minute (BR-06)	SitesClient
Verifies whether a given user can delete it (BR-07)	
Knows the reports it has received	

CommentReport

Responsibilities	Collaborators
Knows its reason, date, reporter id, reported comment, status (pending or resolved) and the id of the administrator who resolved it	Comment
Validates that the reporter is not the author and has not reported that comment before (BR-08)	
Marks itself as resolved after review	

Subscription

Responsibilities	Collaborators
Knows the subscribed citizen id, the construction site id and its creation date	NotificationService

Sends an update to the citizen when the Participation service is informed that the construction site changed (FR_06, BR-14)	

PetitionTemplate *(non-persistent)*

Responsibilities	**Collaborators**
Knows the required sections: recipient entity, requester contact, request, reasons, evidence and response channel (FR_15)	SitesClient
Pre-fills itself with the data of the construction site and its entity, and with the requester data (FR_15)	IdentityClient
Includes the notice that it does not constitute an official filing (BR-13)	PdfGenerator
Exports itself as a PDF (FR_14)	

NotificationService *(non-persistent; uses the external email service)*

Responsibilities	**Collaborators**
Sends construction site update emails to subscribers (FR_06)	Subscription
Obtains the email addresses of the subscribers	IdentityClient

PdfGenerator *(non-persistent)*

Responsibilities	**Collaborators**

Generates the PDF data sheet of a construction site (FR_05)	SitesClient
Generates the PDF of the right of petition template (FR_14)	PetitionTemplate

ParticipationEventHandler *(non-persistent; new)*

Responsibilities	**Collaborators**
Receives the notice that the progress of a construction site changed and notifies its subscribers (BR-14)	Subscription
Receives the notice that a construction site was deleted and deletes its comments, reports, ratings and subscriptions	NotificationService
Receives the notice that a user account was deleted and deletes that user's ratings, comments, reports and subscriptions	Comment
	Rating

SitesClient *(non-persistent; new)*

Responsibilities	**Collaborators**
Checks that a construction site exists	Construction Sites service (REST API)
Obtains the data of a construction site and its responsible entity	

IdentityClient *(non-persistent; new)*

Responsibilities	Collaborators
Obtains the data of a user for the petition template	Identity service (REST API)
Obtains the email addresses of a list of users	

3. Class Diagram

The class diagram is organized by microservice. Figure 1 shows the three services and the classes each one owns; Figures 2, 3 and 4 detail the Identity, Construction Sites and Participation services. Composition is used only between classes of the same service. When a class needs an object owned by another service, it stores that object's id (for example, *constructionSiteId* or *citizenId*) instead of an association, and the client classes (*ParticipationClient*, *SitesClient* and *IdentityClient*) encapsulate the REST calls between services.

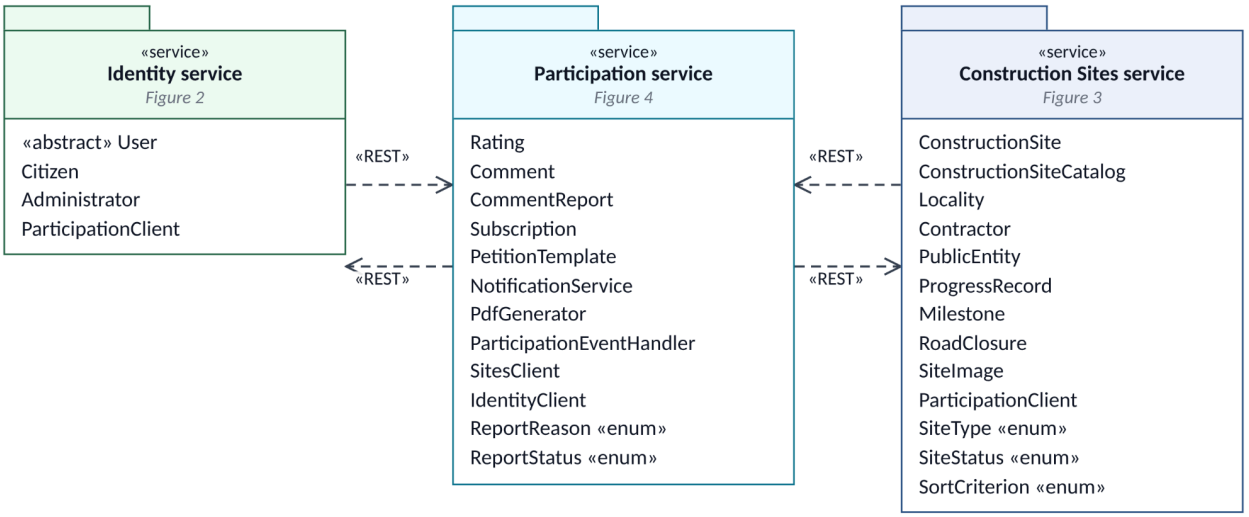


Figure 1

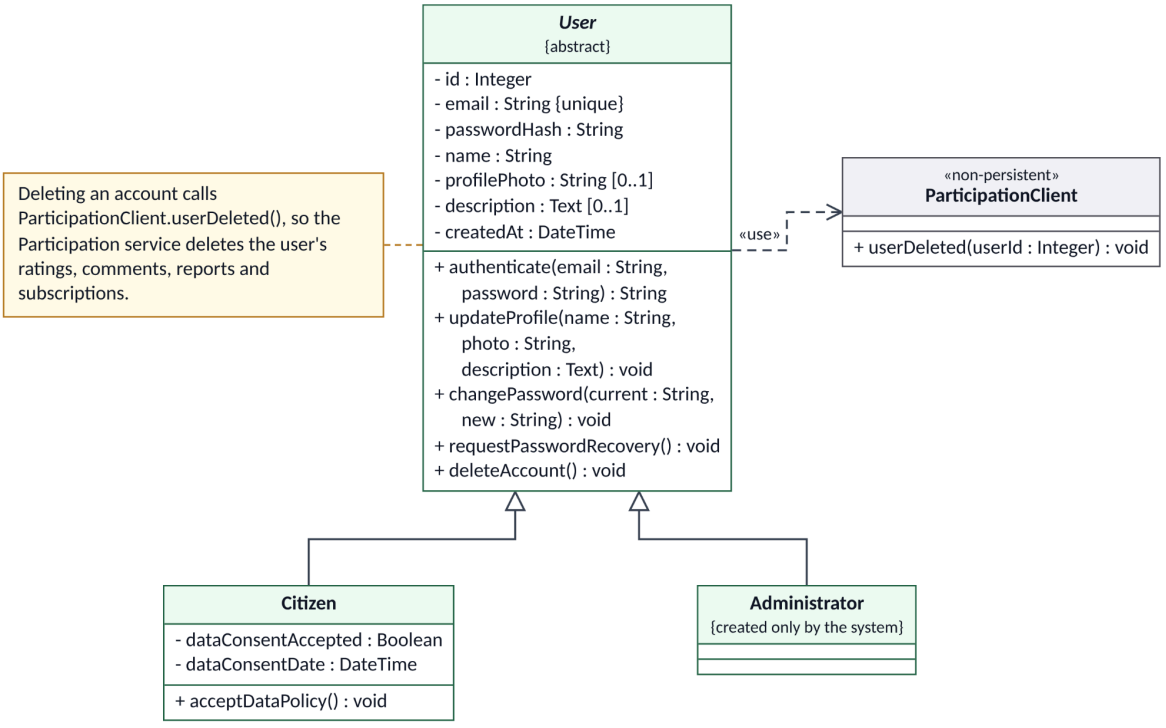


Figure 2

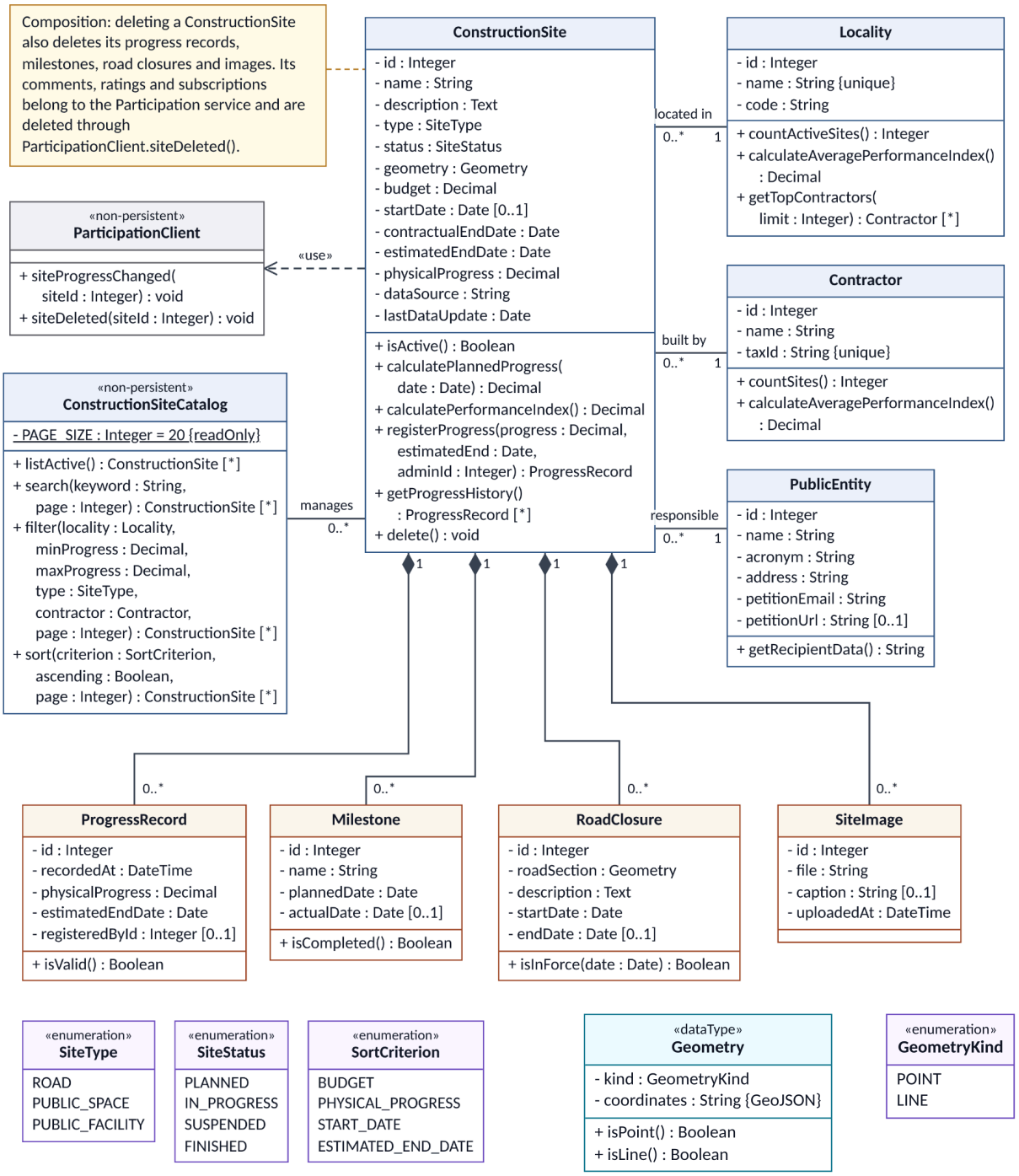

Composition: deleting a ConstructionSite also deletes its progress records, milestones, road closures and images. Its comments, ratings and subscriptions belong to the Participation service and are deleted through ParticipationClient.siteDeleted().
ConstructionSite
- id : Integer
- name : String
- description : Text
- type : SiteType
- status : SiteStatus
- geometry : Geometry
- budget : Decimal
- startDate : Date [0..1]
- contractualEndDate : Date
- estimatedEndDate : Date
- physicalProgress : Decimal
- dataSource : String
- lastDataUpdate : Date
+ isActive() : Boolean
+ calculatePlannedProgress(date : Date) : Decimal
+ calculatePerformanceIndex() : Decimal
+ registerProgress(progress : Decimal, estimatedEnd : Date, adminId : Integer) : ProgressRecord
+ getProgressHistory() : ProgressRecord [*]
+ delete() : void
Locality
- id : Integer
- name : String {unique}
- code : String
+ countActiveSites() : Integer
+ calculateAveragePerformanceIndex() : Decimal
+ getTopContractors(limit : Integer) : Contractor [*]
located in
0..*
1
«non-persistent»
ParticipationClient
+ siteProgressChanged(siteId : Integer) : void
+ siteDeleted(siteId : Integer) : void
«use»
Contractor
- id : Integer
- name : String
- taxId : String {unique}
+ countSites() : Integer
+ calculateAveragePerformanceIndex() : Decimal
built by
0..*
1
«non-persistent»
ConstructionSiteCatalog
- PAGE_SIZE : Integer = 20 {readOnly}
+ listActive() : ConstructionSite [*]
+ search(keyword : String, page : Integer) : ConstructionSite [*]
+ filter(locality : Locality, minProgress : Decimal, maxProgress : Decimal, type : SiteType, contractor : Contractor, page : Integer) : ConstructionSite [*]
+ sort(criterion : SortCriterion, ascending : Boolean, page : Integer) : ConstructionSite [*]
manages
0..*
PublicEntity
- id : Integer
- name : String
- acronym : String
- address : String
- petitionEmail : String
- petitionUrl : String [0..1]
+ getRecipientData() : String
responsible
0..*
1
1
1
1
1
0..*
0..*
0..*
0..*
ProgressRecord
- id : Integer
- recordedAt : DateTime
- physicalProgress : Decimal
- estimatedEndDate : Date
- registeredById : Integer [0..1]
+ isValid() : Boolean
Milestone
- id : Integer
- name : String
- plannedDate : Date
- actualDate : Date [0..1]
+ isCompleted() : Boolean
RoadClosure
- id : Integer
- roadSection : Geometry
- description : Text
- startDate : Date
- endDate : Date [0..1]
+ isInForce(date : Date) : Boolean
SiteImage
- id : Integer
- file : String
- caption : String [0..1]
- uploadedAt : DateTime
«enumeration»
SiteType
ROAD
PUBLIC_SPACE
PUBLIC_FACILITY
«enumeration»
SiteStatus
PLANNED
IN_PROGRESS
SUSPENDED
FINISHED
«enumeration»
SortCriterion
BUDGET
PHYSICAL_PROGRESS
START_DATE
ESTIMATED_END_DATE
«dataType»
Geometry
- kind : GeometryKind
- coordinates : String {GeoJSON}
+ isPoint() : Boolean
+ isLine() : Boolean
«enumeration»
GeometryKind
POINT
LINE

Figure 3

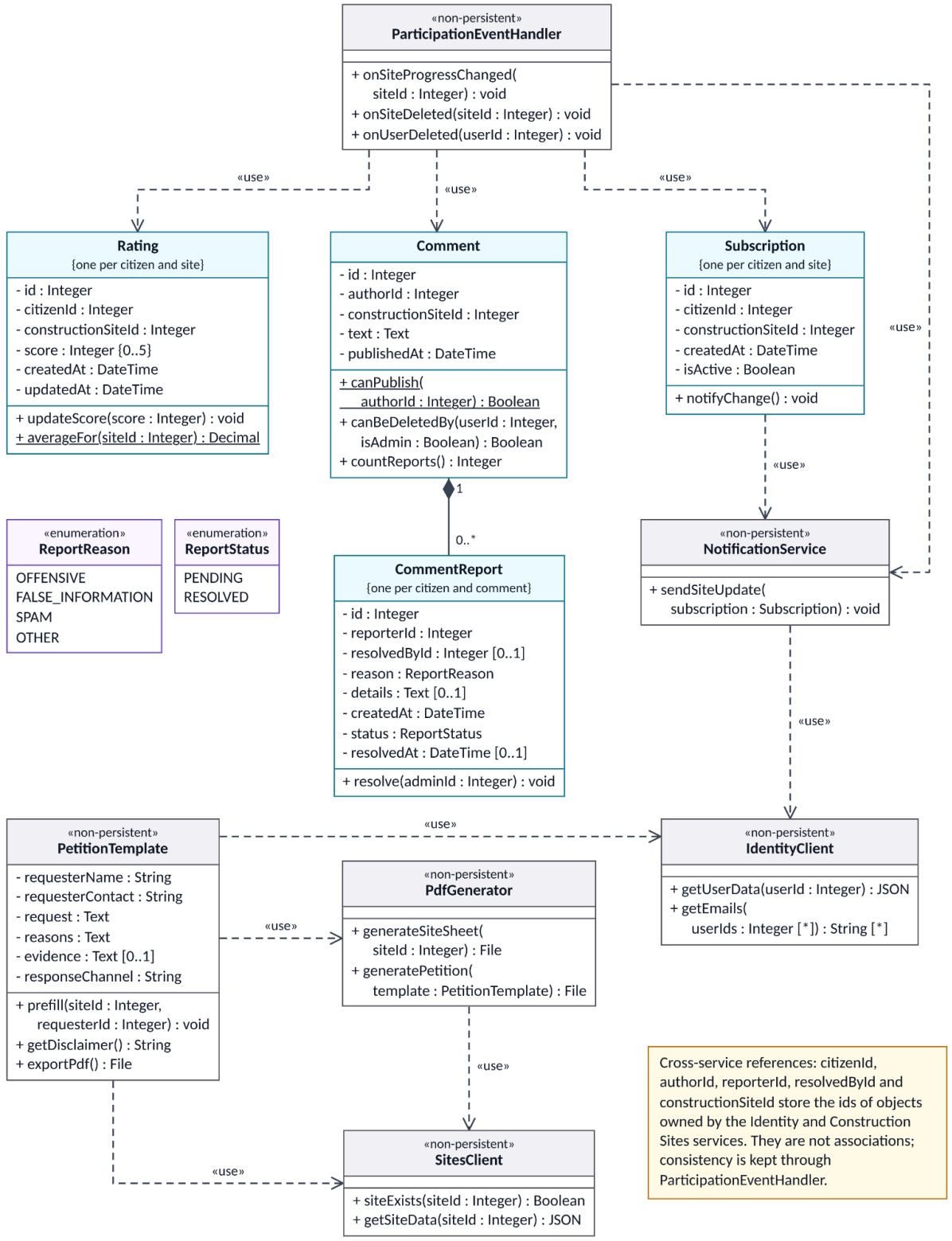

«non-persistent»
ParticipationEventHandler
+ onSiteProgressChanged(
siteId : Integer) : void
+ onSiteDeleted(siteId : Integer) : void
+ onUserDeleted(userId : Integer) : void
«use»
«use»
«use»
Rating
{one per citizen and site}
- id : Integer
- citizenId : Integer
- constructionSiteId : Integer
- score : Integer {0..5}
- createdAt : DateTime
- updatedAt : DateTime
+ updateScore(score : Integer) : void
+ averageFor(siteId : Integer) : Decimal
Comment
- id : Integer
- authorId : Integer
- constructionSiteId : Integer
- text : Text
- publishedAt : DateTime
+ canPublish(
authorId : Integer) : Boolean
+ canBeDeletedBy(userId : Integer,
isAdmin : Boolean) : Boolean
+ countReports() : Integer
Subscription
{one per citizen and site}
- id : Integer
- citizenId : Integer
- constructionSiteId : Integer
- createdAt : DateTime
- isActive : Boolean
+ notifyChange() : void
«use»
1
0..*
«use»
«enumeration»
ReportReason
OFFENSIVE
FALSE_INFORMATION
SPAM
OTHER
«enumeration»
ReportStatus
PENDING
RESOLVED
CommentReport
{one per citizen and comment}
- id : Integer
- reporterId : Integer
- resolvedById : Integer [0..1]
- reason : ReportReason
- details : Text [0..1]
- createdAt : DateTime
- status : ReportStatus
- resolvedAt : DateTime [0..1]
+ resolve(adminId : Integer) : void
«non-persistent»
NotificationService
+ sendSiteUpdate(
subscription : Subscription) : void
«use»
«use»
«non-persistent»
PetitionTemplate
- requesterName : String
- requesterContact : String
- request : Text
- reasons : Text
- evidence : Text [0..1]
- responseChannel : String
+ prefill(siteId : Integer,
requesterId : Integer) : void
+ getDisclaimer() : String
+ exportPdf() : File
«non-persistent»
IdentityClient
+ getUserData(userId : Integer) : JSON
+ getEmails(
userIds : Integer [*]) : String [*]
«non-persistent»
PdfGenerator
+ generateSiteSheet(
siteId : Integer) : File
+ generatePetition(
template : PetitionTemplate) : File
«use»
«use»
«non-persistent»
SitesClient
+ siteExists(siteId : Integer) : Boolean
+ getSiteData(siteId : Integer) : JSON
«use»
Cross-service references: citizenId, authorId, reporterId, resolvedById and constructionSiteId store the ids of objects owned by the Identity and Construction Sites services. They are not associations; consistency is kept through ParticipationEventHandler.

Figure 4

4. Relational Database Model

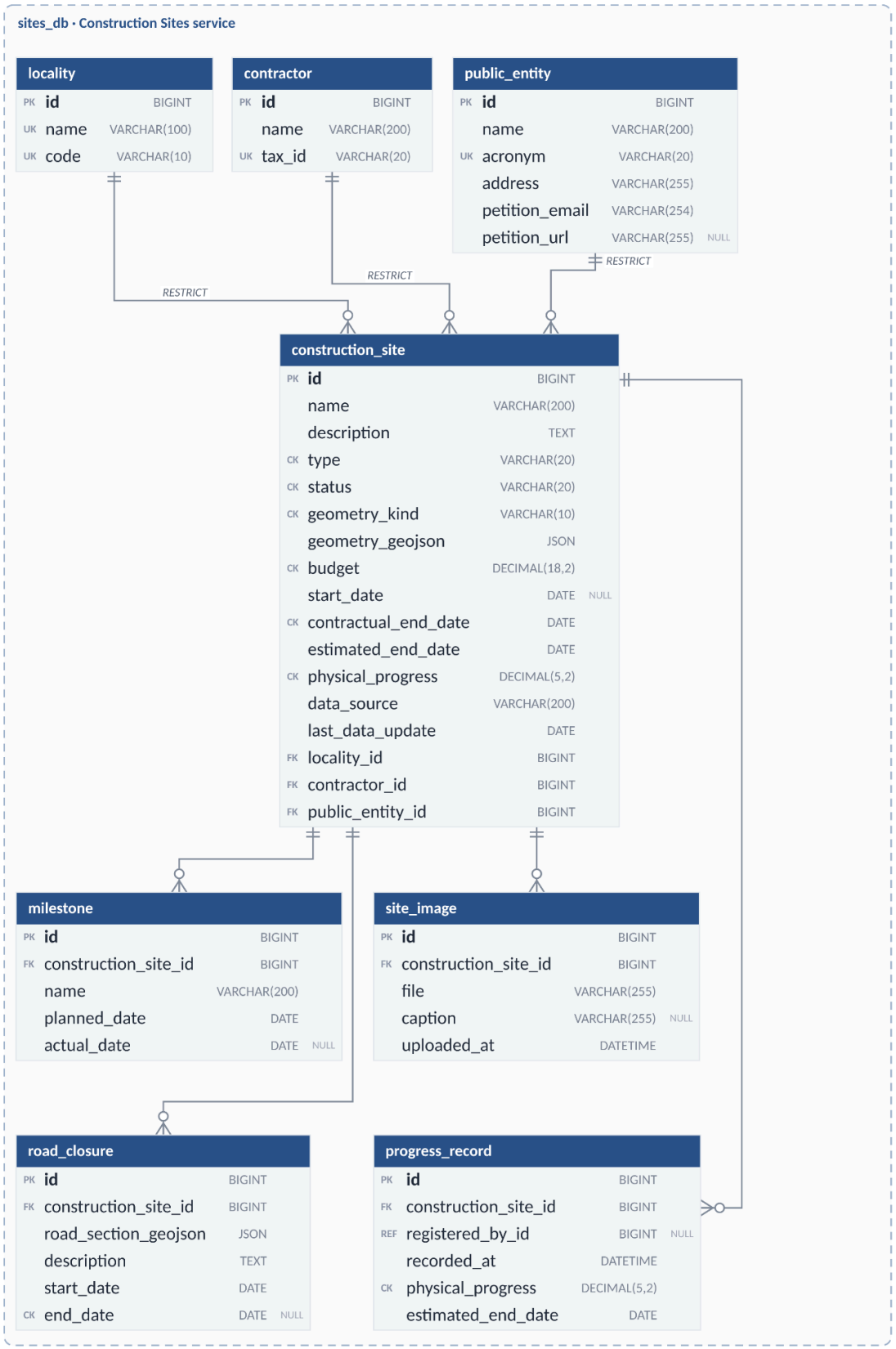


Figure 5



Figure 6

Each service owns its own database: *identity_db* (table *user*), *sites_db* (Figure 5) and *participation_db* (Figure 6). For simplicity, the three databases run on the same MySQL server, but no service reads or writes the database of another service.

Cross-service references. Columns that point to a table of another service are marked as REF: *rating.citizen_id*, *rating.construction_site_id*, *comment.author_id*,

comment.construction_site_id, *comment_report.reporter_id*, *comment_report.resolved_by_id*, *subscription.citizen_id*, *subscription.construction_site_id* and *progress_record.registered_by_id*. They store the id of the referenced row but have no foreign key constraint, because a database cannot reference another database. Foreign keys inside each database keep their ON DELETE actions.

Consistency. Because these references have no ON DELETE CASCADE, deletions are propagated by the services: when a construction site or a user account is deleted, the owning service notifies the Participation service, which deletes the related ratings, comments, comment reports and subscriptions. The system is therefore eventually consistent.

Controlled denormalization. The columns *physical_progress* and *estimated_end_date* of *construction_site* duplicate the values of the most recent *progress_record*. Filtering and sorting by progress (FR_07, FR_13) need the current value in the same table, and *registerProgress()* updates both tables in a single transaction.

CHECK Restrictions

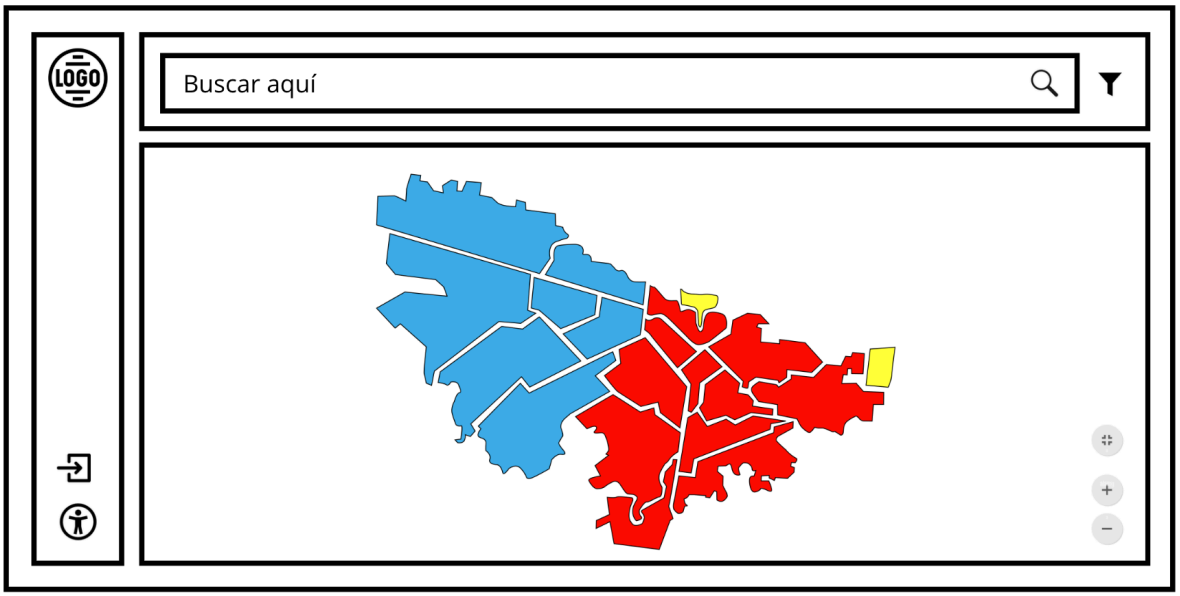
Tabla.columna	Restricción
construction_site.type	IN (‘ROAD’, ‘PUBLIC_SPACE’, ‘PUBLIC_FACILITY’)
construction_site.status	IN (‘PLANNED’, ‘IN_PROGRESS’, ‘SUSPENDED’, ‘FINISHED’)
construction_site.geometry_kind	IN (‘POINT’, ‘LINE’)

comment_report.reason	IN (‘OFFENSIVE’, ‘FALSE_INFORMATION’, ‘SPAM’, ‘OTHER’)
comment_report.status	IN (‘PENDING’, ‘RESOLVED’)
user.role	IN (‘CITIZEN’, ‘ADMIN’)
user.data_consent_accepted	role = ‘ADMIN’ OR data_consent_accepted = TRUE (BR-02)
construction_site.physical_progress, progress_record.physical_progress	BETWEEN 0 AND 100
construction_site.budget	≥ 0
construction_site.contractual_end_date	≥ start_date
road_closure.end_date	IS NULL OR ≥ start_date
rating.score	BETWEEN 0 AND 5 (BR-05)

5. Mockups

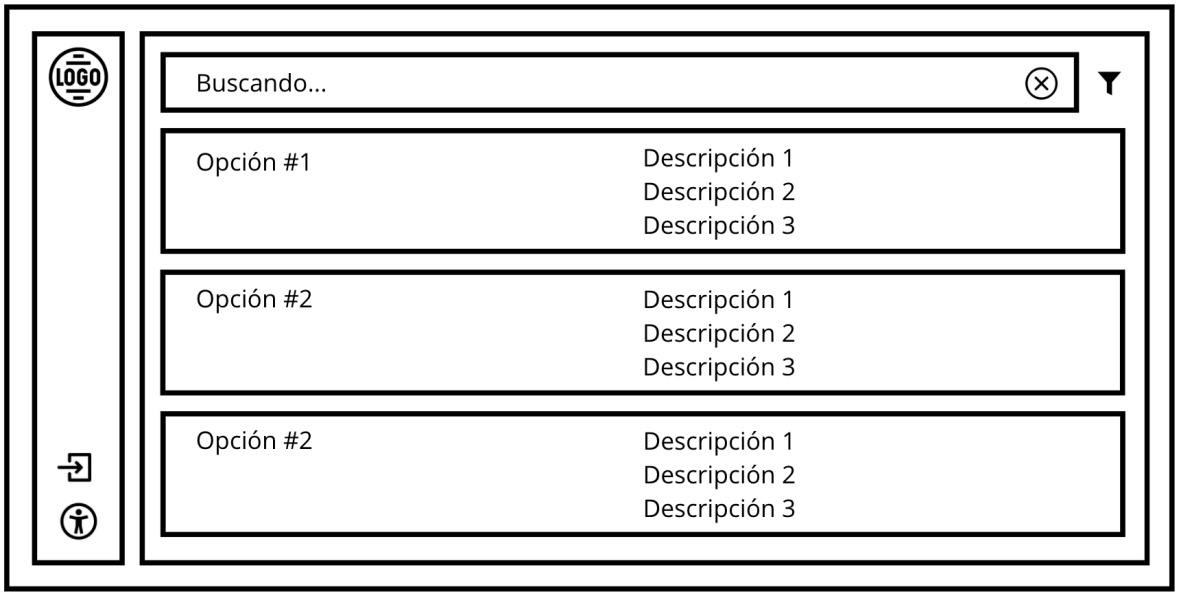
1. Main Page

From the main page, users can navigate the map or use the search engine to locate active public works, log in with their credentials, or customize accessibility settings.



2. **Search**

Users can search for active public works by name or address. Additionally, the filter option allows users to refine their search criteria for more specific results.



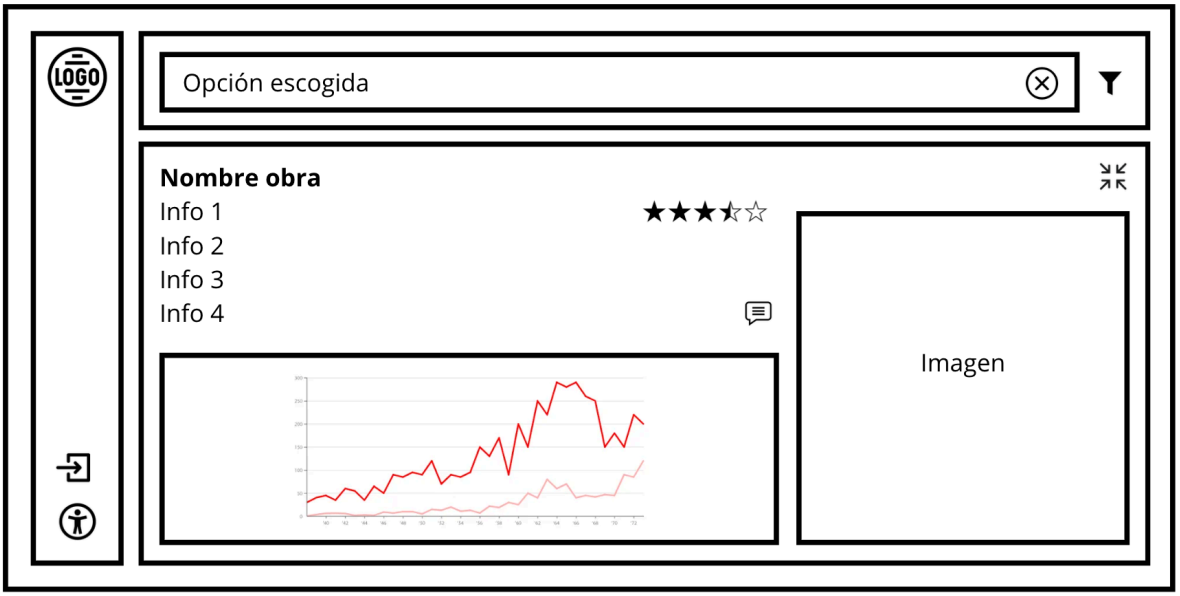
3. **Map**

When the user selects one of the active public works, the system will show a popup window with basic information of the selected work.



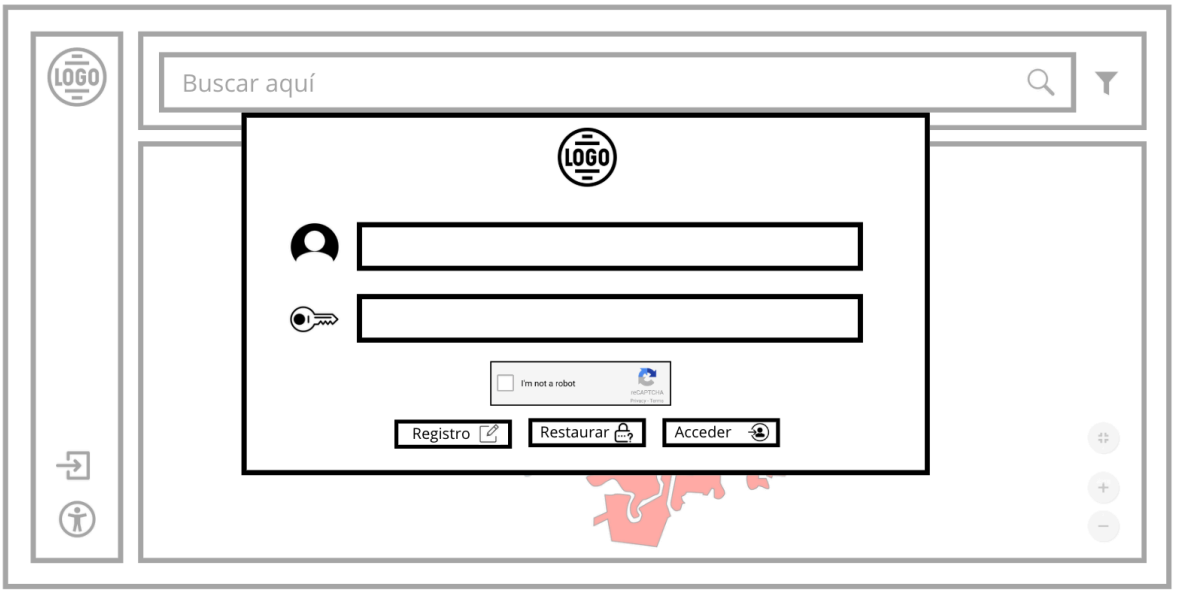
4. Expand information window

When the user clicks the expand window icon, then the system will open the Work information module showing information, rates, comments, progress graph and image of the selected work.



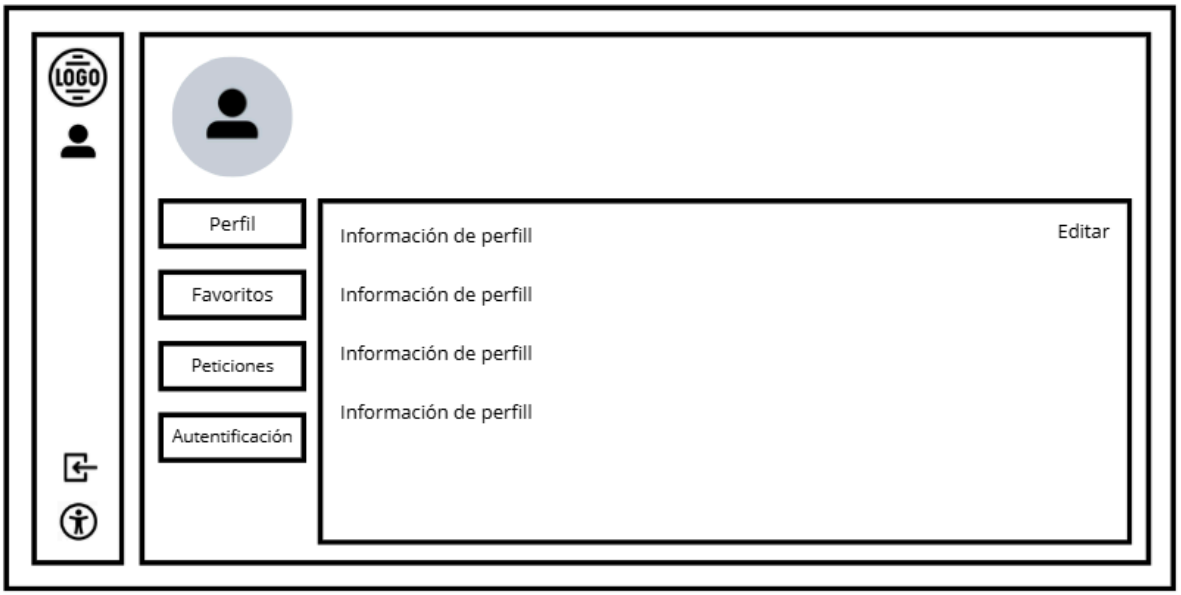
5. Login window

Given that the user desires to access his profile, when the user selects the login icon, then the login popup window will appear, so the user can enter his credentials and select the login button.



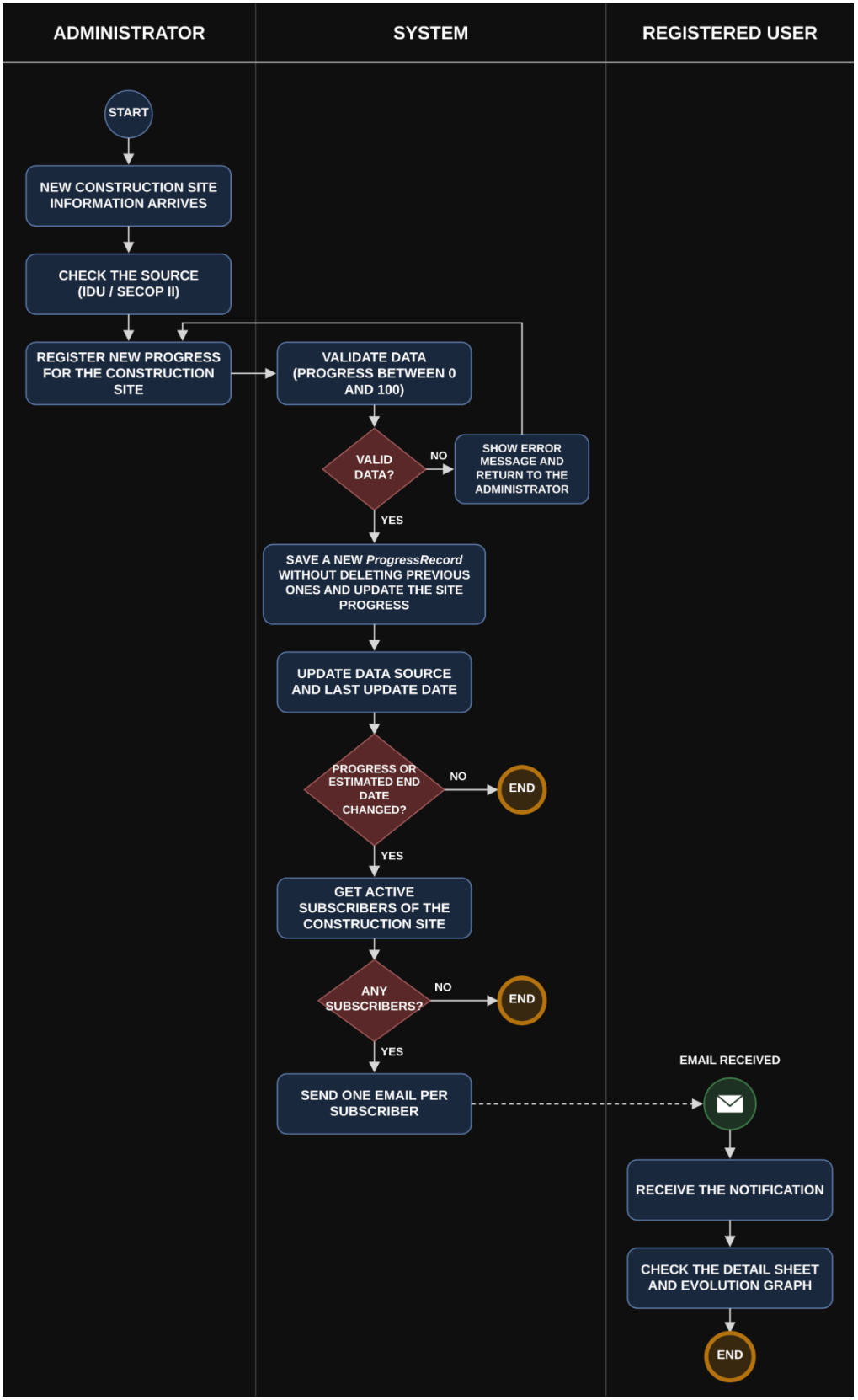
6. Profile page

When the user presses the profile icon, then the system will open the profile window, where the user can access his profile information, saved works on the favorite tab, lawsuits and credentials.

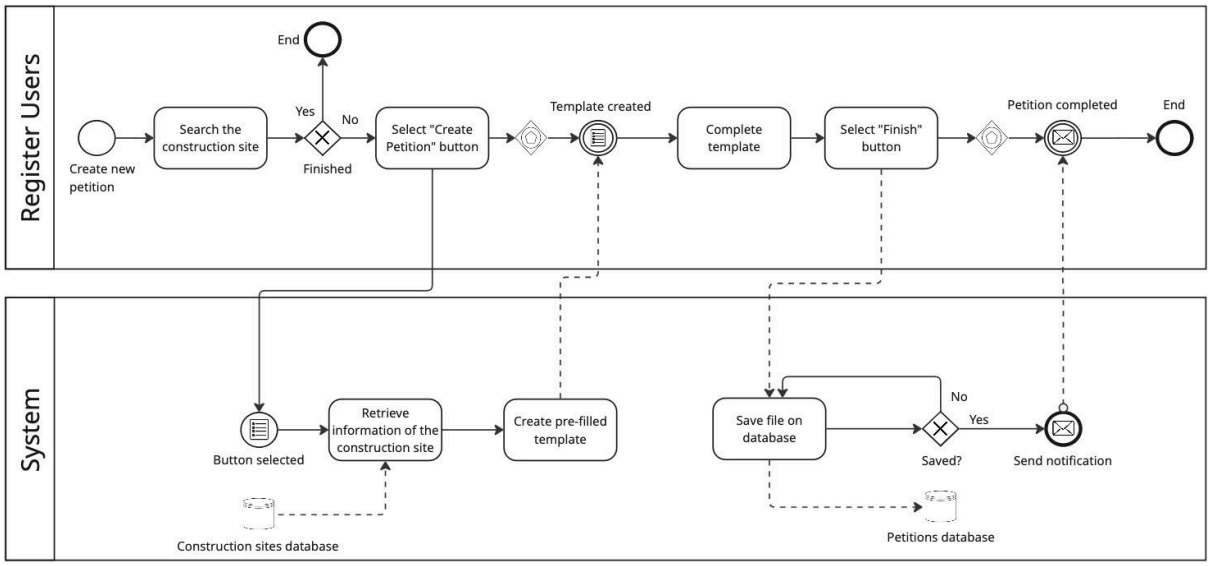


6. Business Model Processes

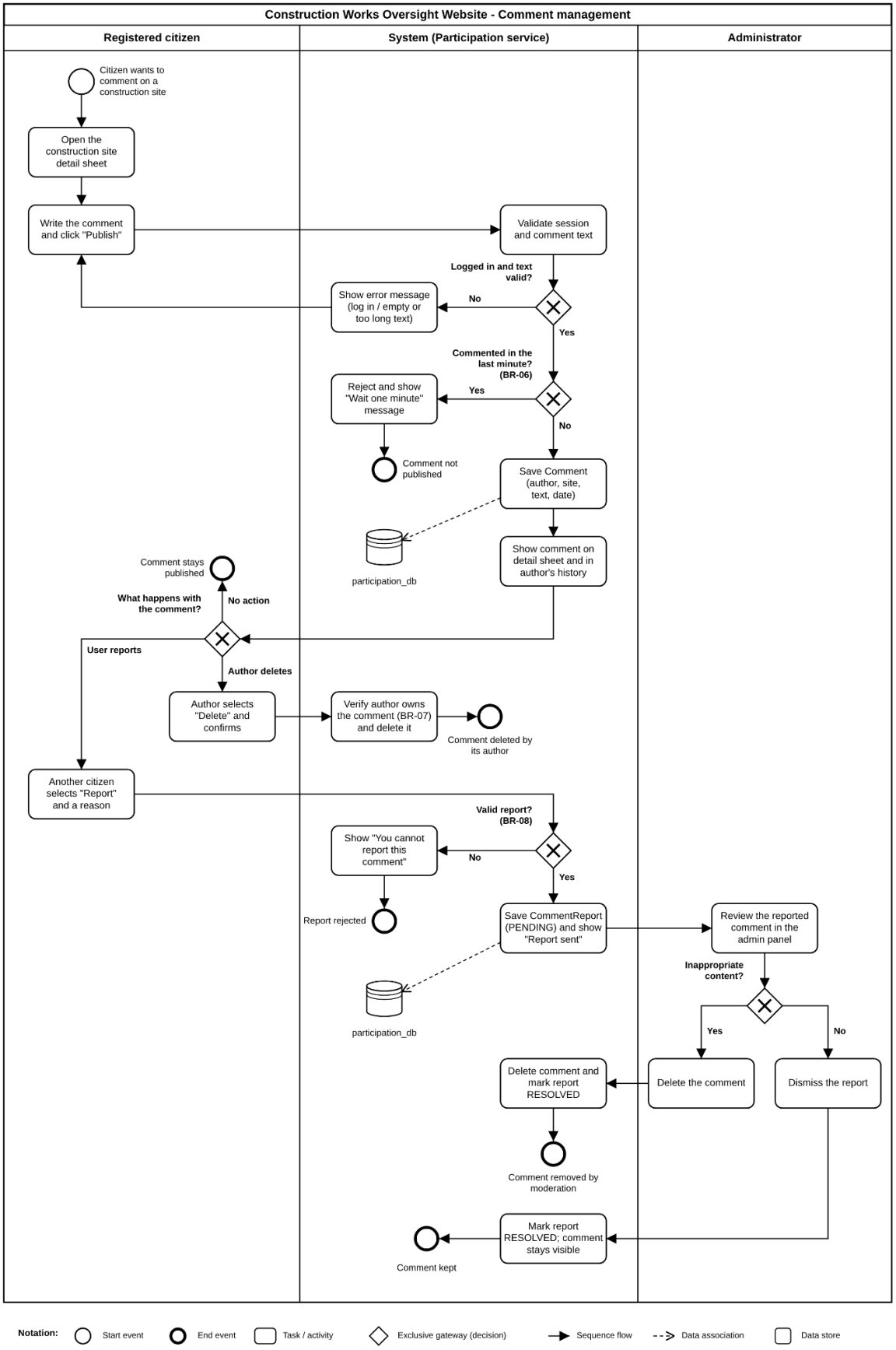
6.1 Project progress update and subscriber notification



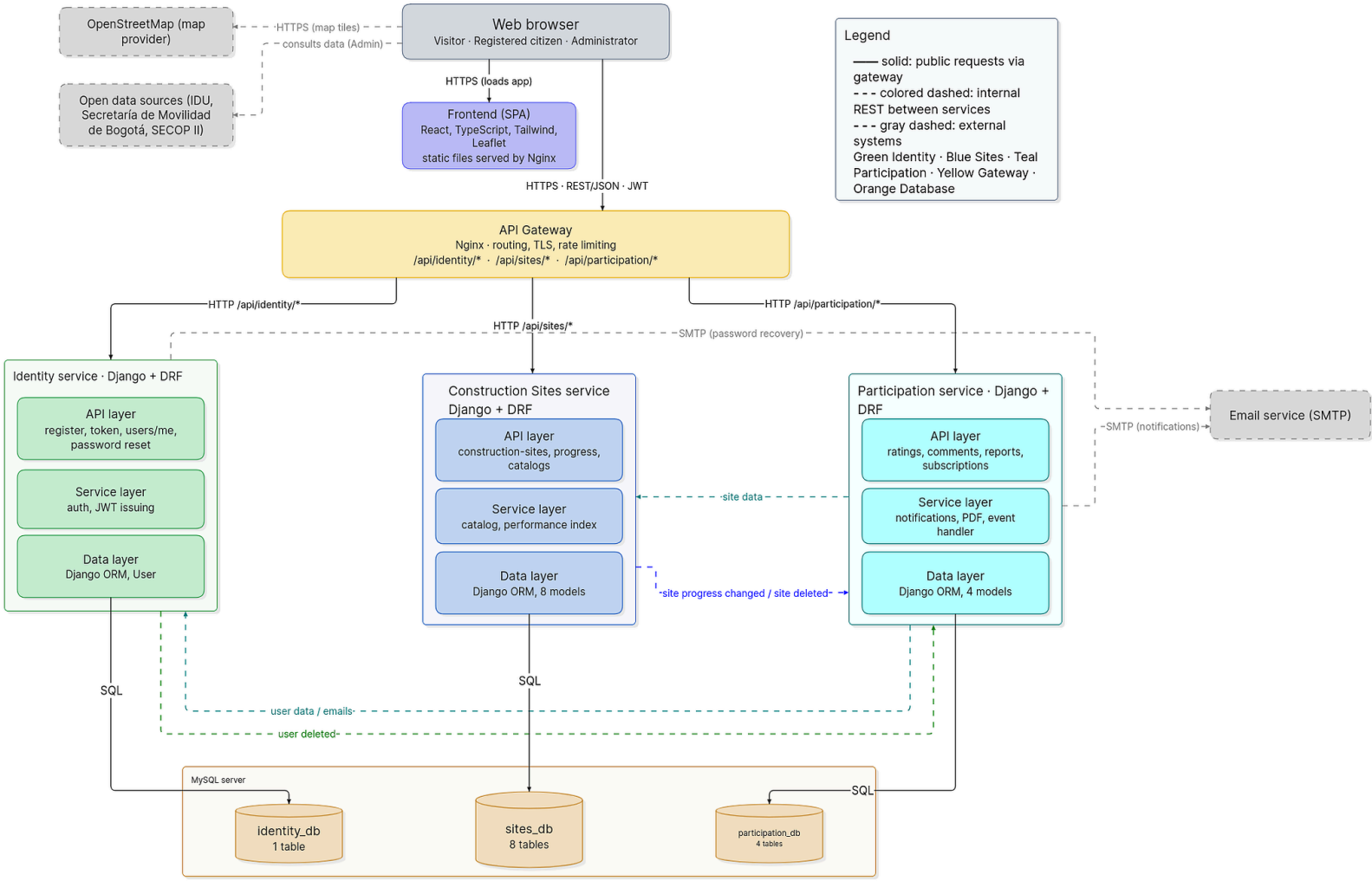
6.2 Create a petition



6.3 Comment management (publish, delete, report and moderate)



7. Architecture Diagram



Declaration of AI use

This workshop was developed with the support of Claude as an assistant. It was used to draft and review parts of the system domain (system context, actor characterization and business rules), to outline the business process, and to plan and review the microservices architecture. All AI-generated content was reviewed, adjusted and approved by the team members, who remain responsible for the final design decisions.

References

1. UML Diagrams. *UML Class and Object Diagrams Overview*. https://www.uml-diagrams.org/class-diagrams-overview.html
2. UML Diagrams. *UML Composite Aggregation (Composition)*. https://www.uml-diagrams.org/composition.html
3. Visual Paradigm. *What is BPMN?* https://www.visual-paradigm.com/guide/bpmn/what-is-bpmn/
4. Lucidchart. *How to Draw Architectural Diagrams*. https://www.lucidchart.com/blog/how-to-draw-architectural-diagrams
5. Richardson, C. *Pattern: API Gateway / Backends for Frontends*. microservices.io. https://microservices.io/patterns/apigateway.html
6. Django Software Foundation. *Django Documentation*. https://docs.djangoproject.com/
7. Encode. *Django REST Framework*. https://www.django-rest-framework.org/
8. Oracle. *MySQL 8.0 Reference Manual — CHECK Constraints*. https://dev.mysql.com/doc/refman/8.0/en/create-table-check-constraints.html
9. Butler, H., et al. (2016). *RFC 7946: The GeoJSON Format*. IETF. https://datatracker.ietf.org/doc/html/rfc7946
10. Congreso de la República de Colombia. (2012). *Ley 1581 de 2012, por la cual se dictan disposiciones generales para la protección de datos personales*.